

Name: Trương Công Hiệp

Class: DS66B

ID: 11247168

Lab 1:

The screenshot shows a PostgreSQL query editor with the following SQL query:

```
--Lab 1|
EXPLAIN (ANALYZE , BUFFERS )
SELECT c . first_name , c . last_name , r . rental_date
FROM customer c
JOIN rental r ON c . customer_id = r . customer_id
WHERE c . customer_id = 1 ;
```

The query plan shows a Nested Loop join. The outer query scans the customer table (Index Scan using customer_pkey on customer c) and the inner query scans the rental table (Seq Scan on rental r). The plan indicates that the join is executed 1 time (loops=1).

QUERY PLAN
1 Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=2.119..7.571 rows=32.00 loops=1)
2 Buffers: shared read=153 dirtied=1
3 -> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.744..0.747 rows=1.00 loops=1)
4 Index Cond: (customer_id = 1)
5 Index Searches: 1
6 Buffers: shared read=3
7 -> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=1.372..6.815 rows=32.00 loops=1)
8 Filter: (customer_id = 1)
9 Rows Removed by Filter: 16012
10 Buffers: shared read=150 dirtied=1
11 Planning:
12 Buffers: shared hit=86 read=4 dirtied=1
13 Planning Time: 6.579 ms
14 Execution Time: 7.601 ms

Total rows: 14 Query complete 00:00:00.083 CRLF Ln 26, Col 8

1. Identify the roles

- Outer (top) table: customer
- Inner (bottom) table: rental

3. Efficiency: Why is loops = 1 in the Inner Table node?

- The loops value indicates how many times the inner plan node is executed during the join.

- Because the outer table (customer) returned only **1 row** (rows=1.00), the query planner only needed to execute the inner loop (Seq Scan on rental) **1 time** (loops=1).

Lab 2:

```

34 --Lab2:
35 EXPLAIN (ANALYZE , BUFFERS )
36 SELECT f . title , l .name as language
37 FROM film f
38 JOIN language l ON f . language_id = l . language_id ;
39

```

Data Output Messages Notifications

Showing rows: 1 to 14 Page No: 1 of 1

Line	QUERY PLAN
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=3.389..3.622 rows=1000.00 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=88 read=1
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.016..0.140 rows=1000.00 loops=1)
5	Buffers: shared hit=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=3.345..3.346 rows=6.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared read=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=3.333..3.334 rows=6.00 loops=1)
10	Buffers: shared read=1
11	Planning:
12	Buffers: shared hit=147 read=1
13	Planning Time: 4.634 ms
14	Execution Time: 3.685 ms

✓ Successfully run. Total query runtime: 82 msec. 14 rows affected. ✕

Total rows: 14 Query complete 00:00:00.082 CRLF Ln 35, Col 1

Windows taskbar: 3:33 PM 9/4/2026

1. Identify Phases

- **Hashed (Built) Table:** language (represented by Seq Scan on language l under the Hash node at line 6 & 9).
- **Probed Table:** film (represented by Seq Scan on film f at line 4).

2. Memory Check

- **Memory Usage: 9kB** (as shown in line 7: Buckets: 1024 Batches: 1 Memory Usage: 9kB).

3. Set work_mem

```

35  --Lab2:
36  EXPLAIN (ANALYZE , BUFFERS )
37  SELECT f . title , l .name as language
38  FROM film f
39  JOIN language l ON f . language_id = l . language_id ;
40  set work_mem='64kB';
41

```

Data Output Messages Notifications



Showing rows: 1 to 12

	QUERY PLAN
	text
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.033..0.313 rows=1000.00 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=89
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.016..0.121 rows=1000.00 loo...
5	Buffers: shared hit=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.011..0.011 rows=6.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared hit=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.006..0.008 rows=6.00 loo...
10	Buffers: shared hit=1
11	Planning Time: 0.180 ms
12	Execution Time: 0.369 ms

Lab3:

Query Query History

```

48 EXPLAIN (ANALYZE , BUFFERS )
49 SELECT f . film_id , f . title , i . inventory_id
50 FROM film f
51 JOIN inventory i ON f . film_id = i . film_id
52 ORDER BY f . film_id ;
53

```

Data Output Messages Notifications

Showing rows: 1 to 18 SQL Page No: 1 of 1

QUERY PLAN	text
1	Sort (cost=566.45..577.90 rows=4581 width=23) (actual time=1.860..2.093 rows=4581.00 loops=1)
2	Sort Key: f.film_id
3	Sort Method: external merge Disk: 192kB
4	Buffers: shared hit=113, temp read=24 written=28
5	-> Hash Join (cost=110.50..193.39 rows=4581 width=23) (actual time=0.230..0.888 rows=4581.00 loops=1)
6	Hash Cond: (i.film_id = f.film_id)
7	Buffers: shared hit=113
8	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.009..0.144 rows=4581.00 loops=1)
9	Buffers: shared hit=25
10	-> Hash (cost=98.00..98.00 rows=1000 width=19) (actual time=0.190..0.191 rows=1000.00 loops=1)
11	Buckets: 1024 Batches: 1 Memory Usage: 59kB
12	Buffers: shared hit=88
13	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=19) (actual time=0.009..0.110 rows=1000.00 loops=1)
14	Buffers: shared hit=88
15	Planning:
16	Buffers: shared hit=6

Total rows: 18 Query complete 00:00:00.079 CRLF Ln 48, Col 1

1. Structure

- No **Sort** node is needed.
- PostgreSQL uses **Index Scan** (or **Index Only Scan**) on **film_id** for both tables to read the rows in pre-sorted order "for free," avoiding an expensive explicit sort operation before executing the Merge Join.

2. Compare

- Yes, it switches back to Hash Join.
- Reason:
 - Without **ORDER BY f.film_id**, PostgreSQL does not need to guarantee a sorted output.
 - For unsorted dataset scans, Hash Join has a lower total estimated cost than Merge Join, as reading via sequential scan and constructing an in-memory hash table is faster than traversing indexes or sorting the tables.

Task1:

```
56 SET enable_indexscan = off;
57 EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24'::timestamp;
58
59 SET enable_indexscan = on;
60 EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24'::timestamp;
61
```

Data Output Messages Notifications

Showing rows: 1 to 7

Page No: 1

QUERY PLAN
text
1 Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.004..0.725 rows=16044.00 loop...
2 Filter: (rental_date > '2005-05-24 00:00:00'::timestamp without time zone)
3 Buffers: shared hit=150
4 Planning:
5 Buffers: shared hit=3
6 Planning Time: 0.056 ms
7 Execution Time: 1.060 ms

Task2:

```
54 --Task2:
55 EXPLAIN (ANALYZE, BUFFERS)
56 SELECT c.customer_id, r.rental_id, i.inventory_id
57 FROM customer c
58 JOIN rental r ON c.customer_id = r.customer_id
59 JOIN inventory i ON r.inventory_id = i.inventory_id;
```

Data Output Messages Notifications

Showing rows: 1 to 22

Page No: 1

of 1

QUERY PLAN
text
Hash Join (cost=168.55..739.57 rows=16044 width=12) (actual time=2.467..21.052 rows=16044.00 loops=1)
Hash Cond: (r.inventory_id = i.inventory_id)
Buffers: shared hit=177 read=7, temp read=40 written=40
-> Hash Join (cost=22.48..375.33 rows=16044 width=12) (actual time=1.440..3.868 rows=16044.00 loops=1)
Hash Cond: (r.customer_id = c.customer_id)
Buffers: shared hit=152 read=7
-> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=10) (actual time=0.018..0.550 rows=16044.00 loop...
Buffers: shared hit=150
-> Hash (cost=14.99..14.99 rows=599 width=4) (actual time=1.414..1.416 rows=599.00 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 30kB
Buffers: shared hit=2 read=7
-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=4) (actual time=0.014..1.350 rows=599.00 loop...
Buffers: shared hit=2 read=7
-> Hash (cost=70.81..70.81 rows=4581 width=4) (actual time=0.835..0.836 rows=4581.00 loops=1)
Buckets: 4096 Batches: 2 Memory Usage: 111kB
Buffers: shared hit=25, temp written=6

Total rows: 22 Query complete 00:00:00.075

CRLF Ln 69, Col 53



ENG 3:44 PM 9/4/2026

70
71
72
73
74

```
ANALYZE customer;  
ANALYZE rental;  
ANALYZE inventory;
```

Data OutputMessagesNotifications

ANALYZE

Query returned successfully in 76 msec.

Total rows:Query complete 00:00:00.076

✓ Query returned successfully in 76 msec. ✕

CRLF Ln 70, Col 1

ENG 3:45 PM
9/4/2026

QueryQuery History

62
63
64
65
66
67
68

```
--Task2:  
EXPLAIN (ANALYZE, BUFFERS)  
SELECT c.customer_id, r.rental_id, i.inventory_id  
FROM customer c  
JOIN rental r ON c.customer_id = r.customer_id  
JOIN inventory i ON r.inventory_id = i.inventory_id
```

Data OutputMessagesNotifications

Showing rows: 1 to 22

Page No: 1 of 1

	QUERY PLAN
1	Hash Join (cost=168.55..739.57 rows=16044 width=12) (actual time=2.367..21.265 rows=16044.00 loops=1)
2	Hash Cond: (r.inventory_id = i.inventory_id)
3	Buffers: shared hit=184, temp read=40 written=40
4	-> Hash Join (cost=22.48..375.33 rows=16044 width=12) (actual time=0.180..2.556 rows=16044.00 loops=1)
5	Hash Cond: (r.customer_id = c.customer_id)
6	Buffers: shared hit=159
7	-> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=10) (actual time=0.020..0.574 rows=16044.00 loops=1)
8	Buffers: shared hit=150
9	-> Hash (cost=14.99..14.99 rows=599 width=4) (actual time=0.149..0.150 rows=599.00 loops=1)
10	Buckets: 1024 Batches: 1 Memory Usage: 30kB
11	Buffers: shared hit=9
12	-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=4) (actual time=0.009..0.067 rows=599.00 loops=1)
13	Buffers: shared hit=9
14	-> Hash (cost=70.81..70.81 rows=4581 width=4) (actual time=1.985..1.986 rows=4581.00 loops=1)
15	Buckets: 4096 Batches: 2 Memory Usage: 111kB
16	Buffers: shared hit=25, temp written=6

Total rows: 22Query complete 00:00:00.083

CRLF Ln 65, Col 1

ENG 3:45 PM
9/4/2026